DAY 3 · SATURDAY 24 OCTOBER

The AI Landscape 2026 & Multi-Agent Systems

Surfaces, models and costs — and when one agent is not enough

Pavlos Protopapas
Harvard University

How to actually use AI in 2026

Two years ago, AI meant a **chat box**.

Today it means systems that write code for hours, browse, book, file, and fix — while you do something else.

75 minutes · Which surface, which model — and when more than one agent?

Outline

1. What changed — reasoning + agents
2. The menu — five ways to use a model
3. Making it work — MCP, costs, choosing
4. Multi-agent systems — patterns, an example, limits

Sixty seconds of history — the only history slide

Pre-2017
ML & deep learning
Statistical pattern recognition; convolutional nets for vision.

2017
Transformers
Self-attention replaces recurrence — compute finally scales.

2022
The ChatGPT moment
Generative chat reaches everyone; the interface era begins.

2023–24
Multimodal
Models see, hear, and speak — vision-language everywhere.

2024–25
Reasoning
Models think before answering; you pay for thinking time.

2025–26
Agents
Tool use, long-horizon tasks, computer operation.

From recognizing patterns → to talking → to seeing → to thinking → to doing.

What may come next — active research directions

NEXT?
Continual learning
Updates from new data or tasks over time—while retaining earlier capabilities.

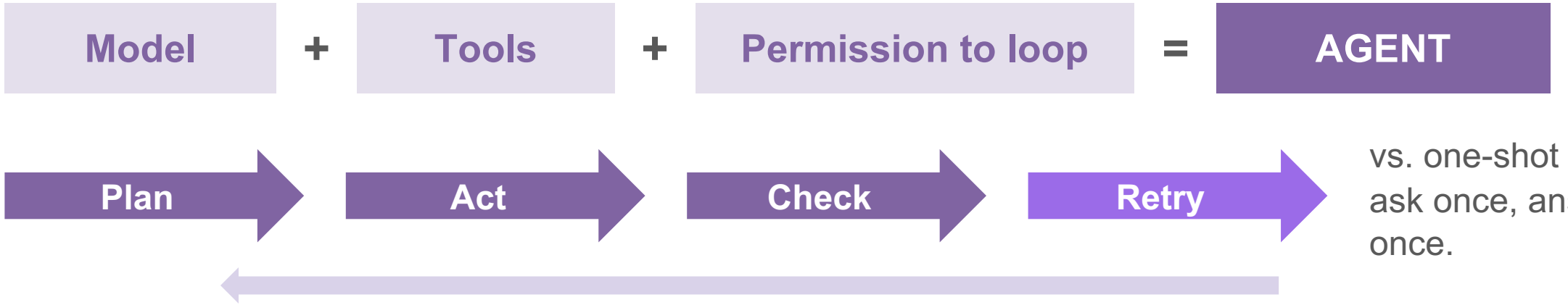
NEXT?
Recursive self-improvement
Models help improve successor models—and the systems that train and serve them.

NEXT?
Embodied intelligence
VLA links vision + language to physical robot actions.

HORIZON
AGI?
Broad, reliable capability across many domains.

Active programs, not promises — overlapping directions, not a guaranteed sequence.

From answers to actions: what an agent actually is



Coding agents

Resolve real GitHub issues end-to-end: write the fix, run tests, open the PR.

Claude Code · Codex · Cursor

Computer-use agents

Operate a desktop with virtual mouse and keyboard across ordinary apps.

OpenAI Computer-Using Agent (CUA)

Research agents

Browse dozens of sources, self-correct, and compile a synthesized brief.

Deep Research modes

From answers to actions: the harness view



…the harness repeats the exchange as needed

ONE GENERAL-PURPOSE AGENT CAN COMBINE THESE CAPABILITIES

Coding capability

Inspect repositories, edit files, run tests, and open pull requests.

shell · files · tests · Git

Computer-use capability

Operate software through screenshots, mouse, and keyboard actions.

computer tool · browser

Research capability

Search, evaluate sources, and synthesize a documented brief.

web · files · code · citations

2023–2026 LLM Landscape



The map: five surfaces, from zero setup to full control

ZERO SETUP — FULL CONTROL

1	2	3	4	5
Chat apps	**Coding agents**	**Direct API / SDK**	**Cloud platforms**	**Local /self-host**
ChatGPT · Claude · Gemini	**Claude Code · Codex · Cursor**	**Lab APIs + official SDKs**	**Bedrock · Foundry · Vertex**	**Ollama · LM Studio · vLLM**
Zero setup, broad knowledge work.	Live in your repo, run commands, open PRs.	Pay per token, your prompt, your tools.	Governed gateway inside your cloud.	Your hardware. Privacy, cost, offline.
Everyone	**Builders & teams**	**Developers**	**Enterprises**	**Privacy & control**

The next few slides walk this map left to right, and you can find a spot for yourself along the way.

Same brand, different doors — don't confuse the products

Every lab ships one model family through several products. Chatbot ≠ Work tier ≠ coding agent ≠ CLI — the model is similar; the surface, permissions, and price change.

Ecosystem	Conversation	Agentic work	Agentic coding	CLI / terminal
OpenAI	ChatGPT (Chat)	ChatGPT Work	Codex	Codex CLI
Anthropic	Claude (Chat)	Claude Cowork	Claude Code	Claude Code CLI
Google	Gemini app	Antigravity		Antigravity CLI
Microsoft	Microsoft Copilot	Microsoft 365 Copilot app	GitHub Copilot	GitHub Copilot CLI
	Surface 1 on the map — chat	**Surface 2 on the map — coding agents**		

Rule of thumb: start with chat for quick questions; agentic work, coding, and CLI increasingly share the same capabilities — their boundaries are now very blurry.

Chat apps — and their hidden depth

No longer “just chat” — what's inside a modern chat app:

Web search
current sources, cited

File analysis
PDFs, sheets, decks

Sandboxed analysis
isolated runtime for code

Canvas / artifacts
in-app editor + preview

Deep research
multi-step sourced reports

Connected apps
your drive, mail, tools

Voice & vision
speech, images + screen

Image generation
GPT Image 2, Nano Banana 2

Projects & memory
persistent context across chats

Scheduled tasks
recurring work, proactive updates

TEAM & ENTERPRISE TIERS

- Admin, access & privacy controls
- Org knowledge & shared workspaces
- Connected-app governance
- Business data excluded from training by default

BEST STARTING POINT

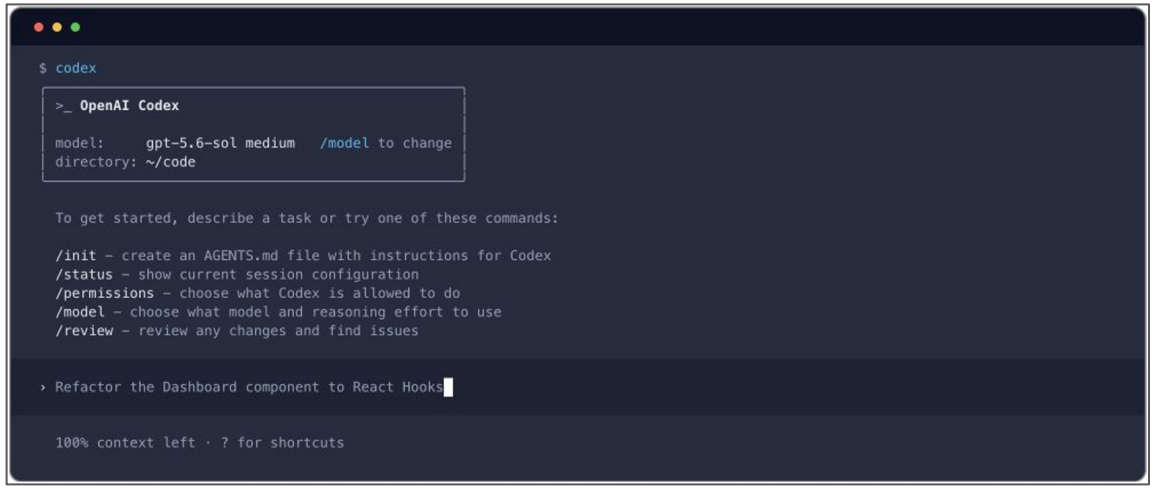
Everyday knowledge work

Start here; move to a specialist agent when the workflow demands it.

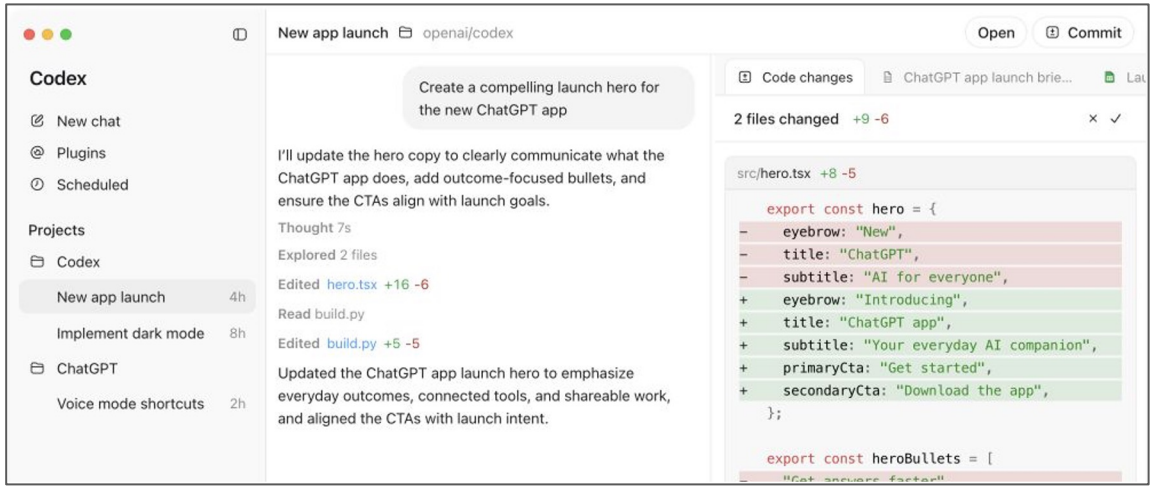
Coding agents — same models, different ergonomics

Codex example: the CLI and desktop app share the same core capabilities.

CODEX CLI



CODEX DESKTOP APP

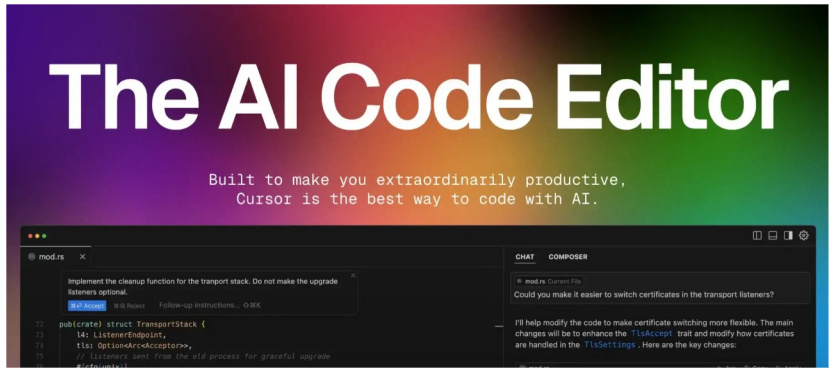


Use the terminal
Type commands or automate repeatable work

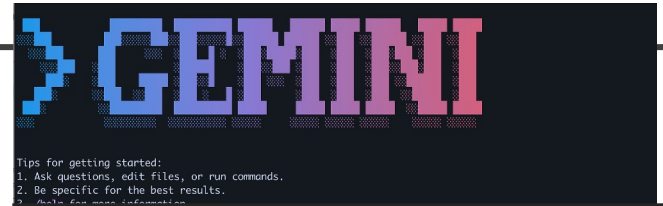
Use the desktop app
Organize tasks and review changes visually

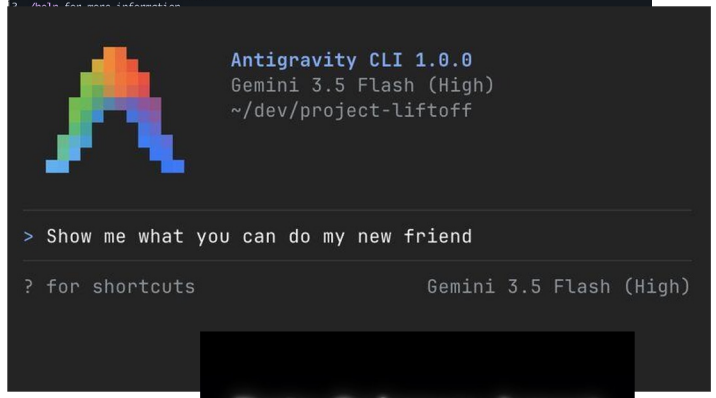
Same core agent. The interface changes how you work.

Agentic LLM Tools: Pair Programming









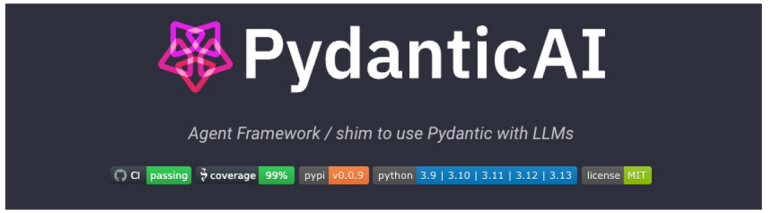


















The API & SDKs — build your own

YOU CONTROL THE REQUEST:

POST /v1/responses

```
{
  "model":     "your-pick",
  "system":    "your rules",
  "tools":     [your_functions],
  "reasoning": "low | high"
}
```

The one concept to land: tokens are the meter

GPT-5.6 Sol example · standard API, short context
USD per 1M tokens; other models and providers vary

Cache hit — reused prompt prefix
$0.50 / 1M

Input — new prompt, context, and tool results
$5.00 / 1M

Output — response + reasoning/thinking tokens
$30.00 / 1M

SUITED FOR

Developers embedding AI into products — anyone needing behavior chat apps can't offer.

Why it matters: a “cheap” model on high reasoning can out-cost an “expensive” one. Budget per completed task, not model price alone.

THREE WAYS:

Raw API — HTTP calls to /v1/… from any language. You handle auth, retries, and streaming.

SDK — official libraries wrapping the same API — typed and batteries included.

Agent SDK — adds tools, session state, and multi-step orchestration.

Cloud platforms — the governed gateway

Same idea, three flavors: many models behind your cloud's IAM, network, audit, and compliance controls.

AWS

Amazon Bedrock

Breadth on AWS

- Managed API access to Anthropic, Meta, Mistral, Cohere, Nova…
- PrivateLink can keep traffic on private AWS paths; IAM controls access
- Agent tooling and guardrails built in

MS

Microsoft Foundry

Microsoft-first

- Enterprise home for Azure OpenAI and other models behind organizational controls
- Deep Entra ID, RBAC, networking, and policy integration
- Natural fit if your organization already runs on Azure

GCP

Gemini Enterprise Agent Platform

Data-stack teams

- Native Gemini with long-context strengths
- BigQuery integration for data-grounded workflows
- Agent Runtime for managed agent deployment

Rule of thumb: your cloud often picks your platform. Suited for enterprises moving from prototype to production.

Running models locally — why and how

Privacy / air-gap	No provider per-token bill	Full runtime control	Works offline

Ollama **The default**

One-command CLI; runs as a background service with an OpenAI-compatible local API.

LM Studio **For non-terminal users**

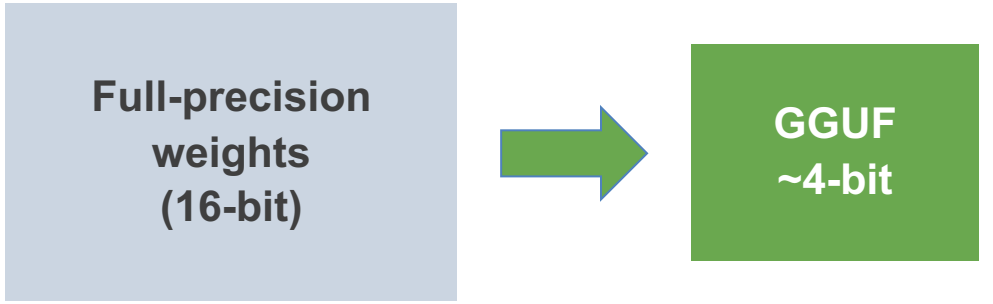
Friendly desktop GUI with a visual model browser and tuning controls.

vLLM **For production serving**

High-throughput engine when many users share one self-hosted model.

Under the hood of most of these: llama.cpp.

The trick that makes it possible: quantization

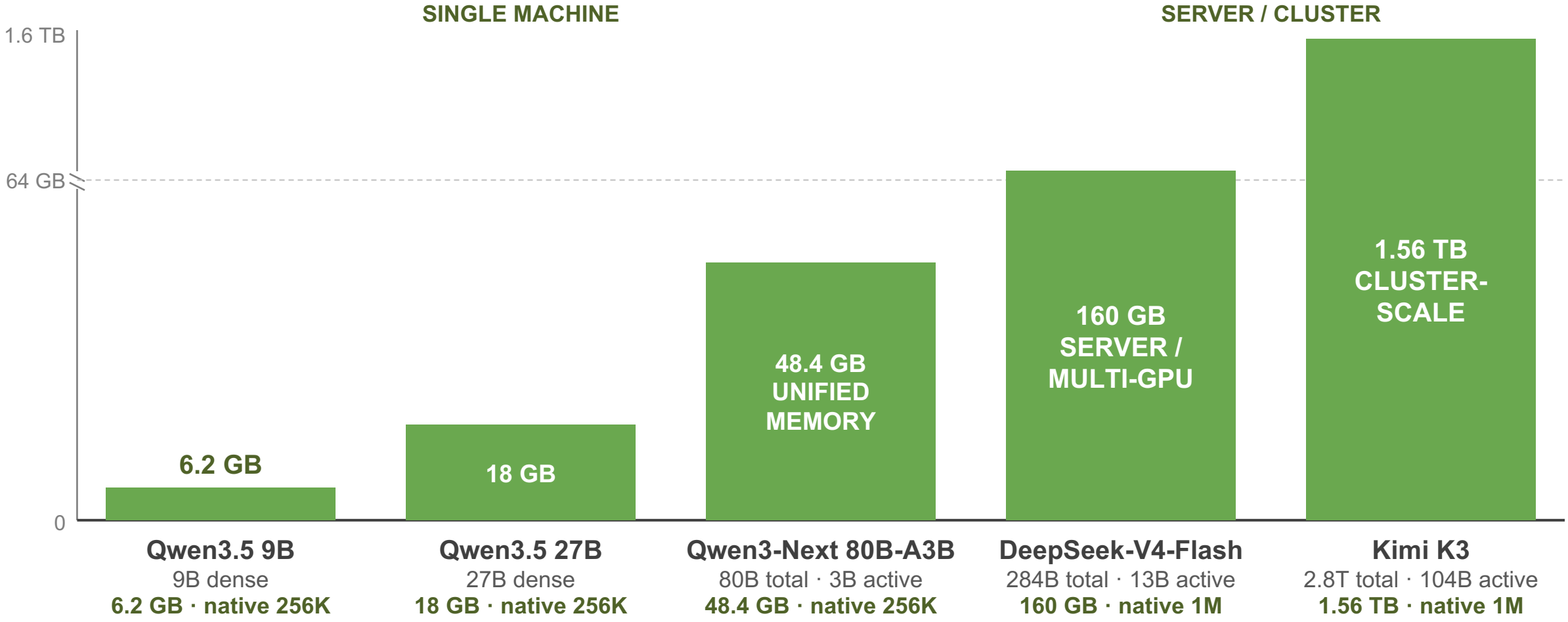


Often ~70–75% smaller than 16-bit weights

Honest caveat: frontier-scale open models can require server-class hardware. Consumer systems usually run the smaller quantized tier.

The hardware ladder — what fits on what

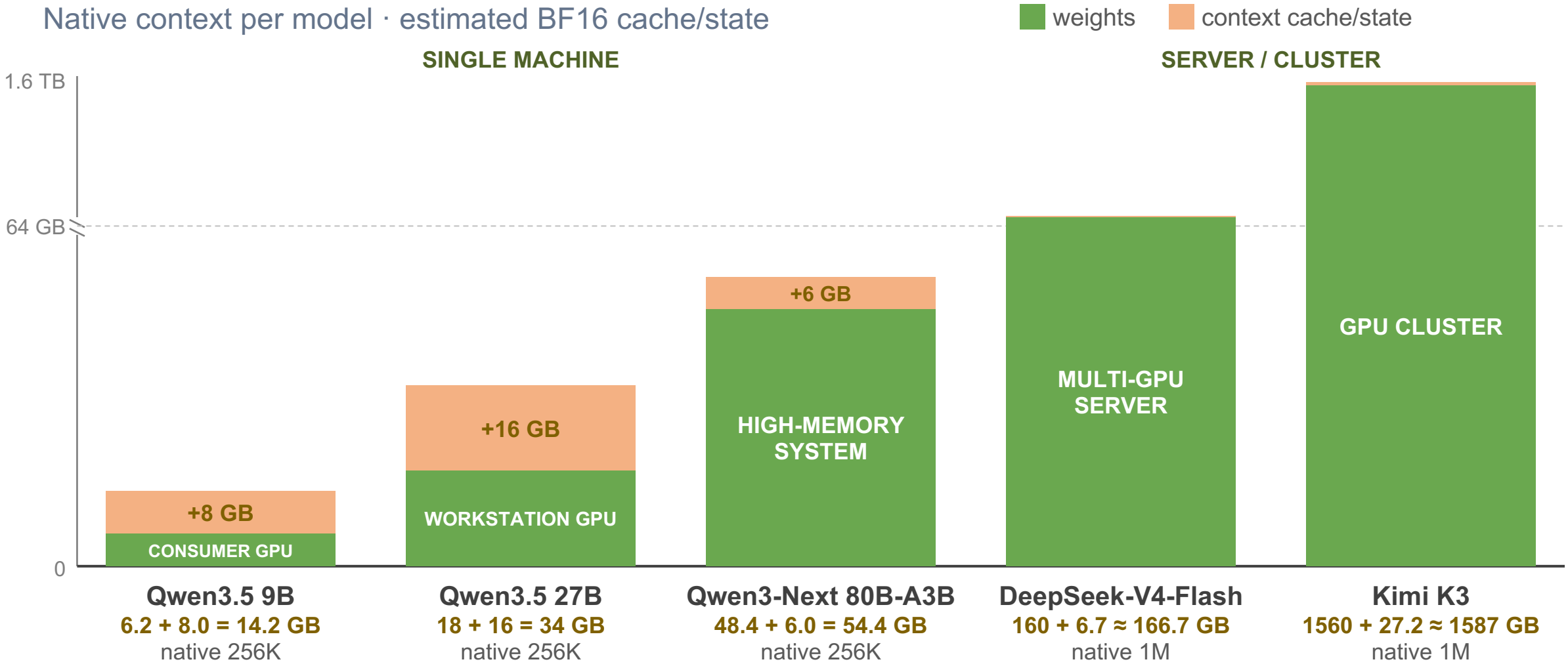
Memory for published low-precision weights only · roughly 4-bit · scale break after 64 GB



Apple Silicon: high-memory systems use unified memory, but context state and runtime buffers share that same pool.

The runtime ladder — what actually has to fit

Native context per model · estimated BF16 cache/state



Native-window capacity view: one sequence per model; concurrency and serving-engine buffers still require headroom.

The open-weight story — why local is even possible

The registry

Hugging Face is where open models live. One nuance: open-weight ≠ open-source (MIT/Apache vs. community licenses).

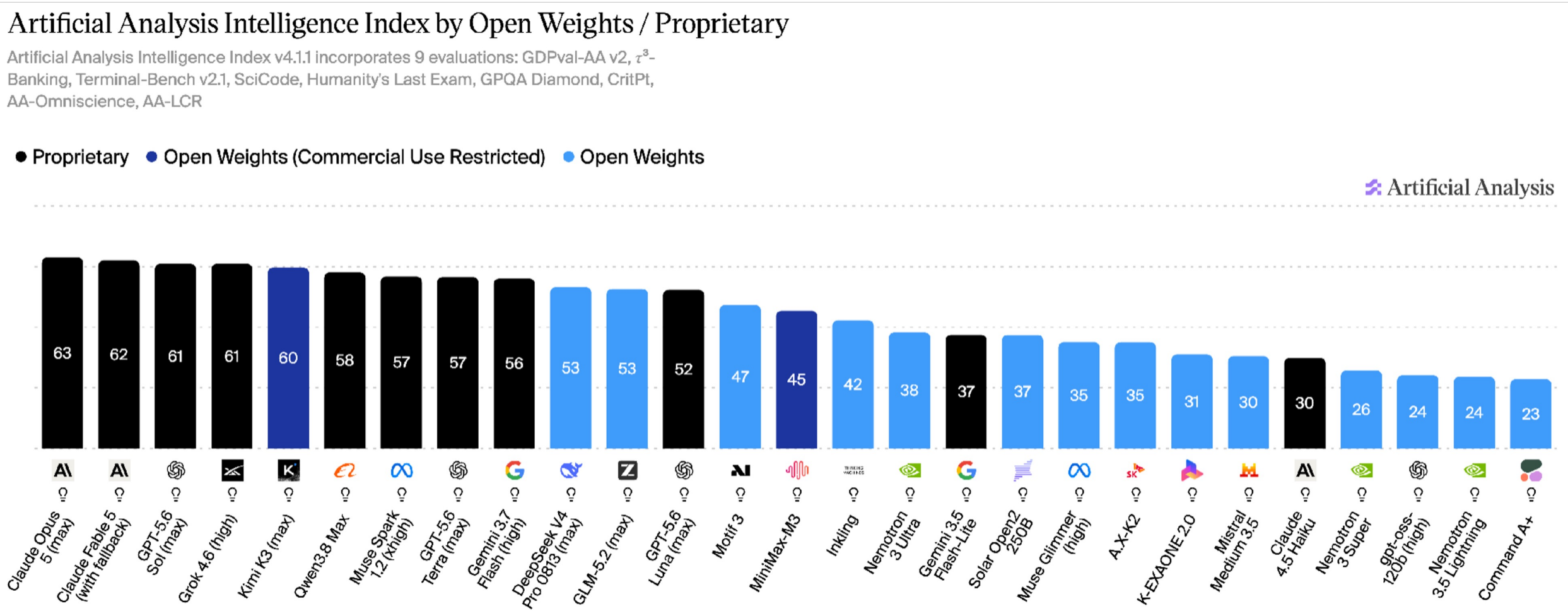
The 2026 headline

Open models sit within a few points of the closed frontier — and the leaders are increasingly Chinese labs: DeepSeek, Qwen, Kimi, GLM. A genuine geopolitical shift.

The DeepSeek sentence

DeepSeek-V4 pairs frontier-class performance with hybrid compressed attention (CSA + HCA) and the Muon optimizer—and continues DeepSeek's industry-wide pressure on API pricing.

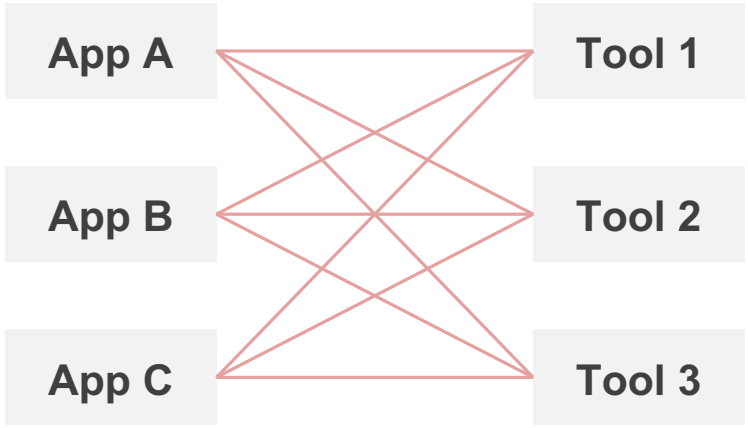
The Great Divide: Proprietary vs. Open Source LLMs



Source: Artificial Analysis, AUG 2026

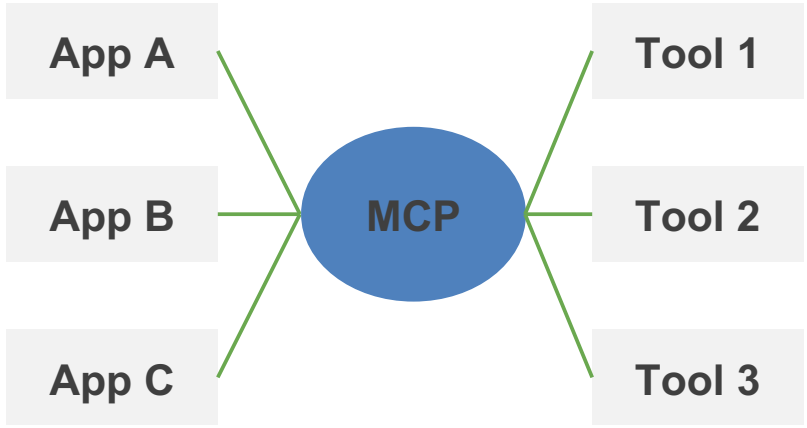
MCP — the USB-C of AI

BEFORE — the N×M problem



9 custom integrations — rebuilt every time

AFTER — one protocol



Expose each system once — any model can use it

10,000+
public MCP servers

~97M
monthly SDK downloads

Linux Foundation
stewarded since Dec 2025

All major labs
OpenAI · Google · Microsoft · AWS

What it means: build your integrations once as MCP servers, and stop rebuilding them every time the model leaderboard flips.

Security beat: new connective tissue = new attack surface (tool poisoning, prompt injection via tool results). Treat MCP servers like any third-party software supply chain.

What it costs — and the three levers that cut it

~100×

price spread between micro-models and frontier reasoning models — and prices keep falling

$ per million tokens →



1 RIGHT-SIZE THE MODEL **BOTH WORKLOADS**

Use the least expensive model that reliably clears the task.

INTERACTIVE AGENT

2 PROMPT CACHING

Reuse stable instructions, tools, and context.

Batch usually does not fit; calls depend on prior results.

ASYNC BACKEND

3 BATCH APIs

Independent, delay-tolerant jobs.

Cache when a stable prefix repeats.

Combine the levers that fit the workload—not every workload can use all three effectively.

How to compare models in 90 seconds

IGNORE — saturated & contaminated

~~MMLU~~ *headline scores above 90% mean nothing*

~~HumanEval~~ *headline scores above 90% mean nothing*

~~GSM8K~~ *headline scores above 90% mean nothing*

Test sets leaked into training data; every flagship aces them. Ignore headlines built on these.

WATCH — execution-based, agentic

SWE-bench *did the agent fix a real GitHub issue?*

TAU-bench *did the database actually change correctly?*

OSWorld *can it operate a real computer?*

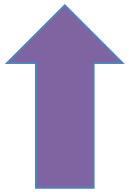
Verify trajectories, not vibes: judge what the agent did, not what it said.

When frontier models are clustered within a few points of each other. **Stop asking "which is smartest" — ask "which clears the bar for my task at the lowest cost."**

The decision matrix — photograph this slide

Your situation	Reach for	Examples
Individual knowledge work	Chat app (paid tier)	Claude · ChatGPT · Gemini
Team with shared data & governance	Enterprise chat tier + connectors	Claude for Work · ChatGPT Enterprise
Software engineering	Coding agent (CLI + IDE)	Claude Code · Codex · Cursor
Building an AI product	Direct API + model routing	Lab APIs, model cascade
Regulated enterprise at scale	Cloud platform	Bedrock · Foundry · Vertex
Hard privacy / offline / high volume	Local or self-hosted open weights	Ollama + Qwen/Gemma · vLLM

What to watch over the next 12 months



Agents leave the repo

From coding to general computer and office work — cross-app workflows on ordinary desktops.



Open weights keep closing in

Pricing pressure continues as open models track the frontier — intelligence trends toward a utility.



Standards & governance mature

MCP spec updates, agent-security tooling, and growing regulatory attention — a frontier model was even briefly suspended by regulators in 2026, a first.

Which surface, which model, for which job?

1 Chat apps | 2 Coding agents | 3 Direct API | 4 Cloud platforms | 5 Local / self-host

1

Pick your surface

Match the map to your workflows, data boundaries, and security needs.

2

Standardize on MCP

Build integrations once; stop rebuilding when the leaderboard flips.

3

Measure cost per task

Not price per token — routing, caching, and batching do the rest.

Part 2 — Multi-agent systems

Composing agents: the patterns, a worked example, and when one agent is enough

Agent Systems

Single Agent System

Consist of a single agent that execute flows in loops

Multiple Agent System

Consist workflow executions distributed across multiple agents

Single Agent Systems

- Consists of a single LLM with instructions + tools
- Handle multiple tasks by adding tools incrementally
- Agent complexity is managed by controlling number of tools
- Evaluation and maintenance is manageable

Multi Agent Systems

- Consists of a workflow across multiple agents
- Multi-agents can be modeled as graphs, with nodes as agents
- Manager Orchestration Pattern (Agents as tools)
  - A manager agent coordinates multiple agents using tool calls
  - Each agent handles very specialized tasks
  - Edges in the graph represent tool calls
- Decentralized Orchestration Pattern (Agent 2 Agent)
  - Agents operate as peers handing over tasks to one another based on specialization
  - Edges in the graph represent handoffs

Multi-agent patterns — the map

1

Orchestrator / worker

- One agent owns the goal, splits it, delegates
- Workers are called like tools and return results
- Example: the Cheese Quiz Guild — 6 specialists + 1 orchestrator

2

Planner / executor

- A planner writes the step list first
- An executor runs each step; replan on failure
- Fits long, structured tasks with checkpoints

3

Handoffs

- Control passes to a specialist agent
- The conversation moves with it — no return
- Example: triage → billing agent (SDK handoff)

4

Agent-to-agent (A2A)

- Separate systems talk over a protocol
- Discovery via agent cards; no shared memory
- Cross-vendor; one slide today, no code

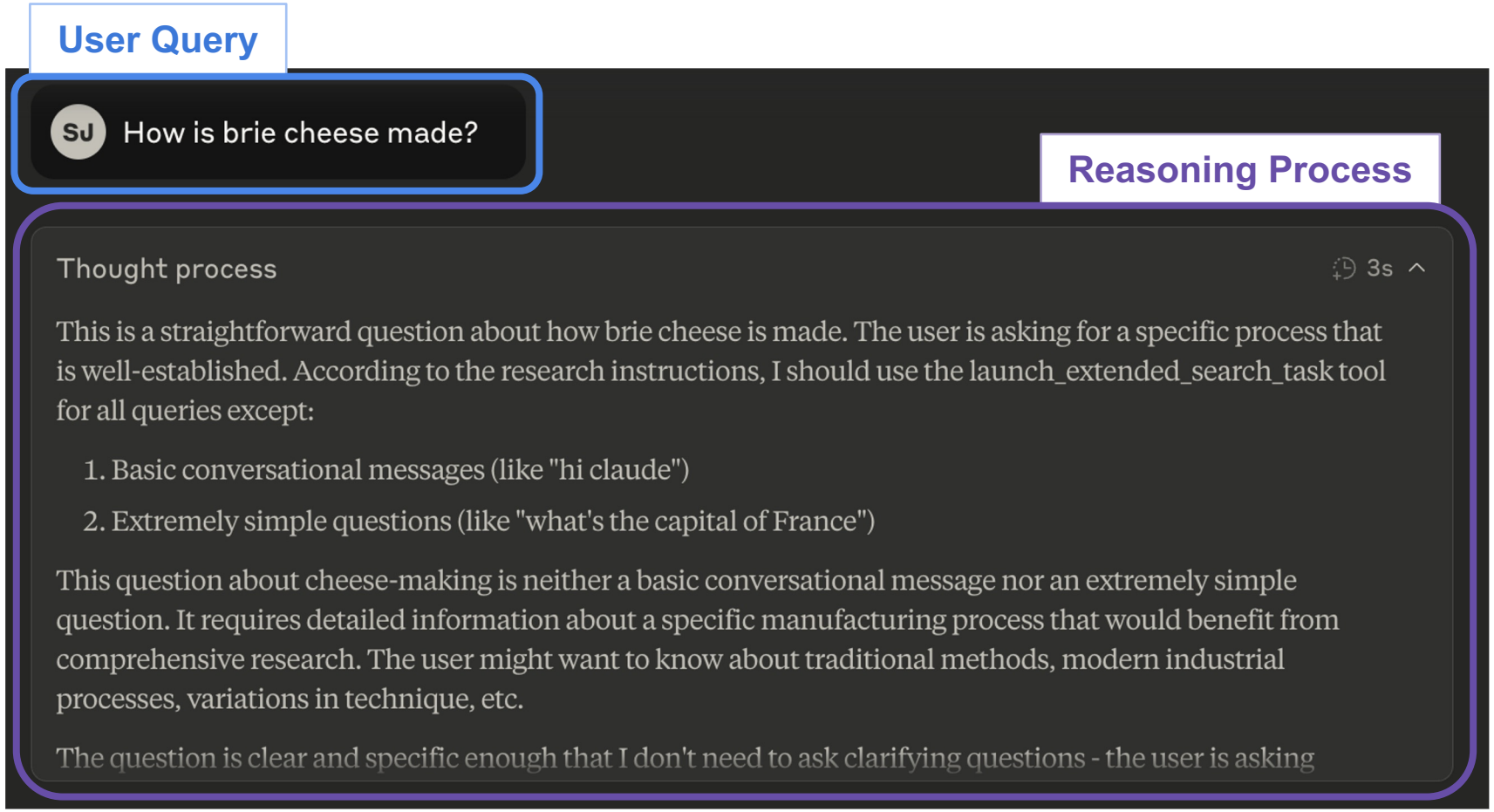
All four share one control question: **who decides the next step — the orchestrator, the plan, the specialist, or a remote peer?**

Today's lab builds pattern 1 with agents-as-tools; handoffs are shown as the alternative.

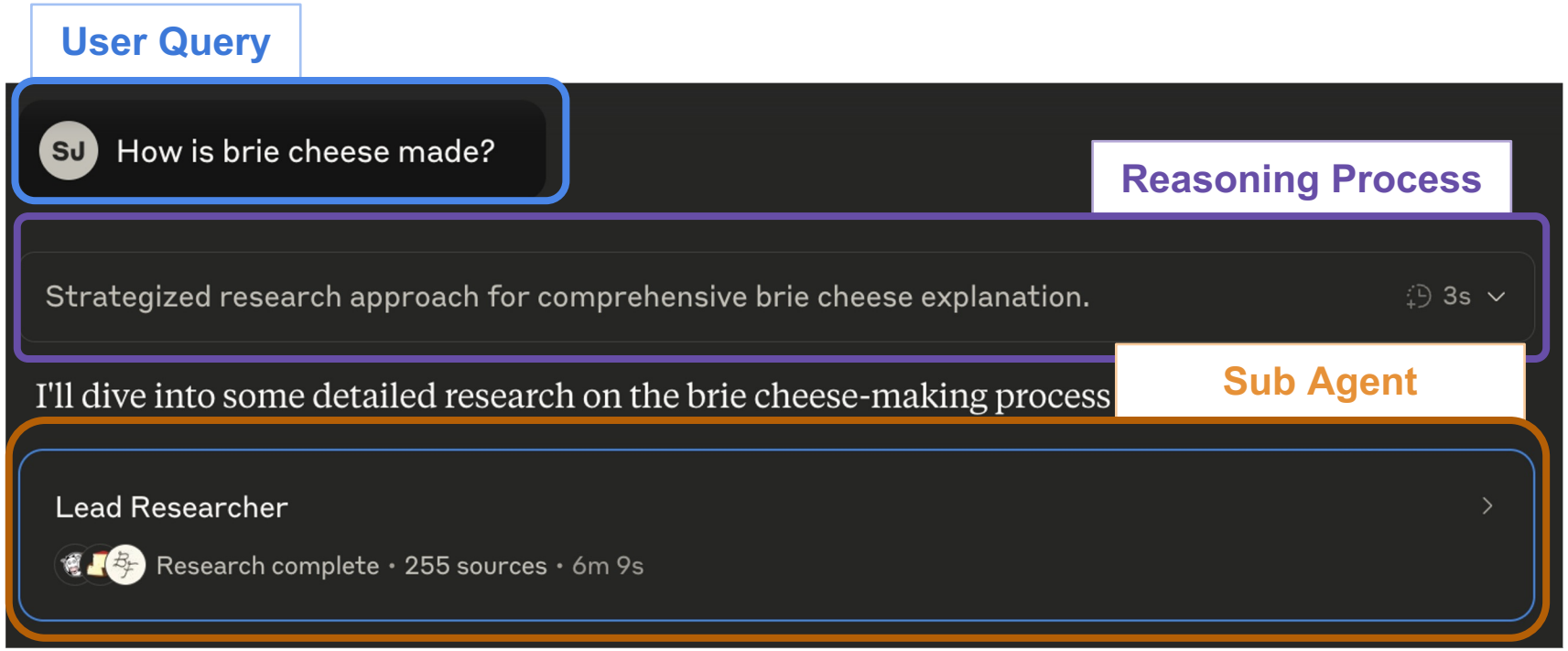
Agent Example: Claude



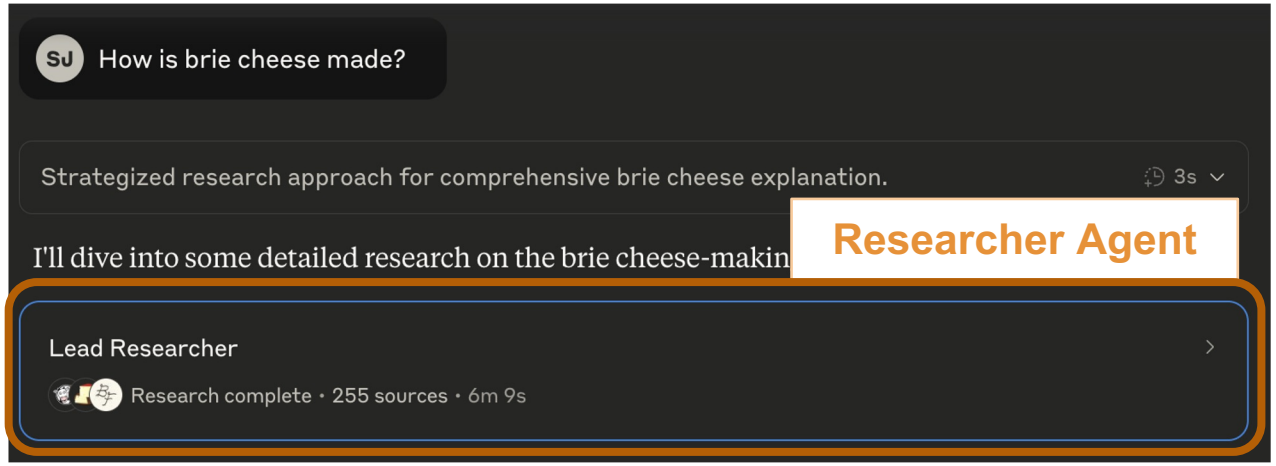
Agent Example: Claude

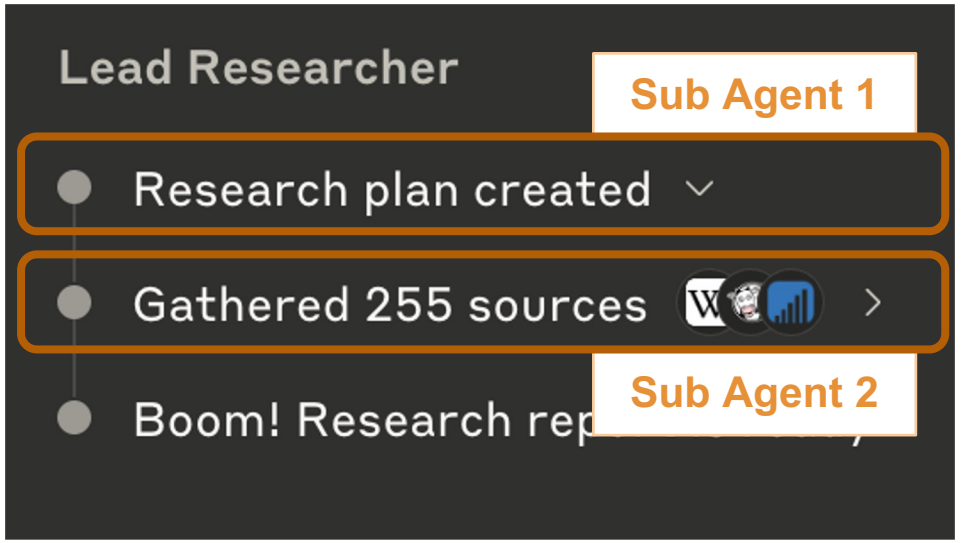


Agent Example: Claude



Agent Example: Claude





Agent Example: Claude

Research Planner Agent

Research plan created

The user is asking for a comprehensive explanation of how brie cheese is made. This is a detailed technical question about a specific food production process. Let me analyze this using the research process guidelines.

1. **Assessment and breakdown**:
2. **Query type determination**: This appears to be a **depth-first query** because:
3. **Detailed research plan development**: For this depth-first query, I should deploy multiple subagents to explore different aspects:

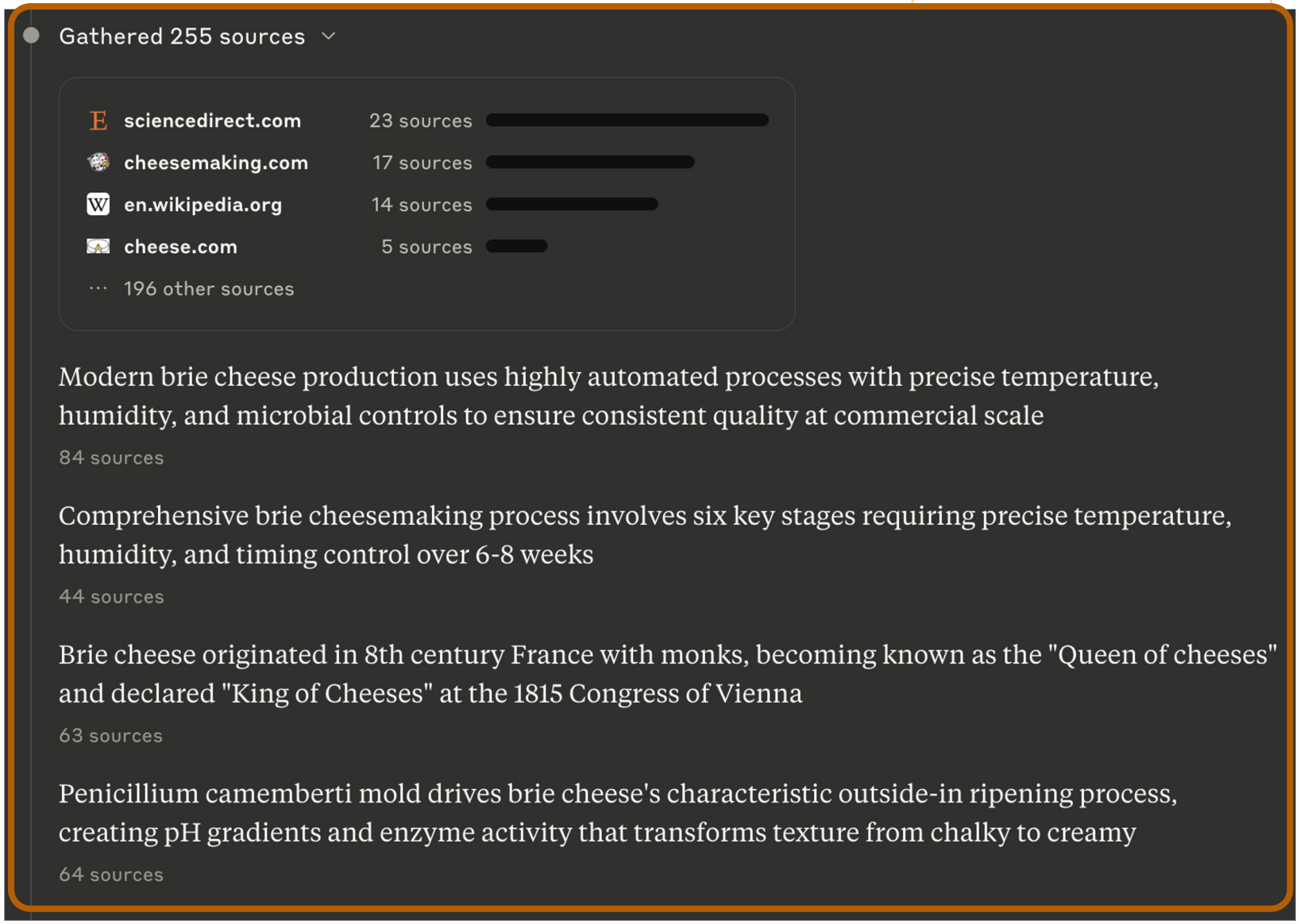
Instructions for more Sub Agents

I should create about 3-4 subagents to cover these different angles comprehensively.

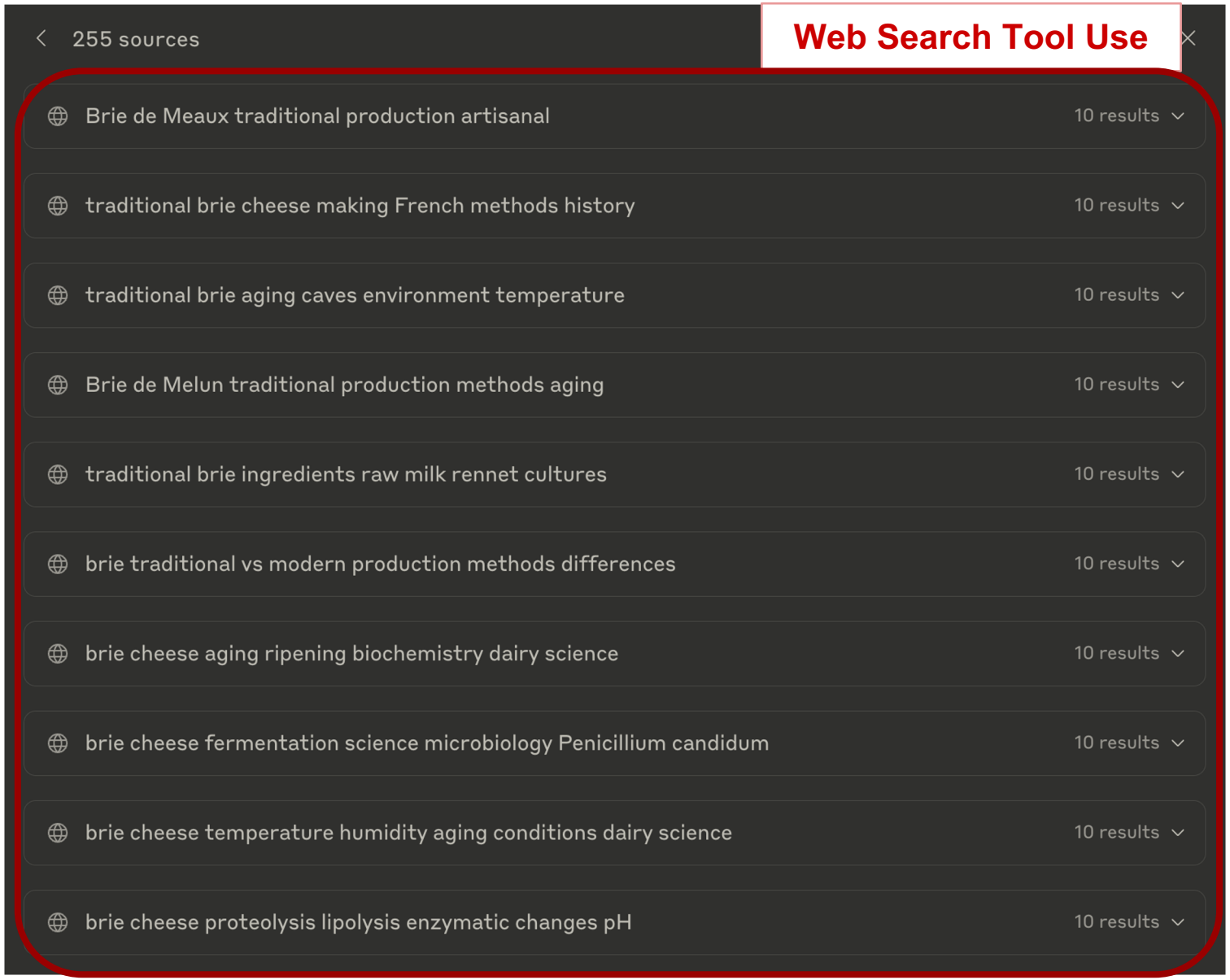
4. **Execution**: Deploy subagents in parallel to research different aspects, then synthesize into a comprehensive report.

Agent Example: Claude

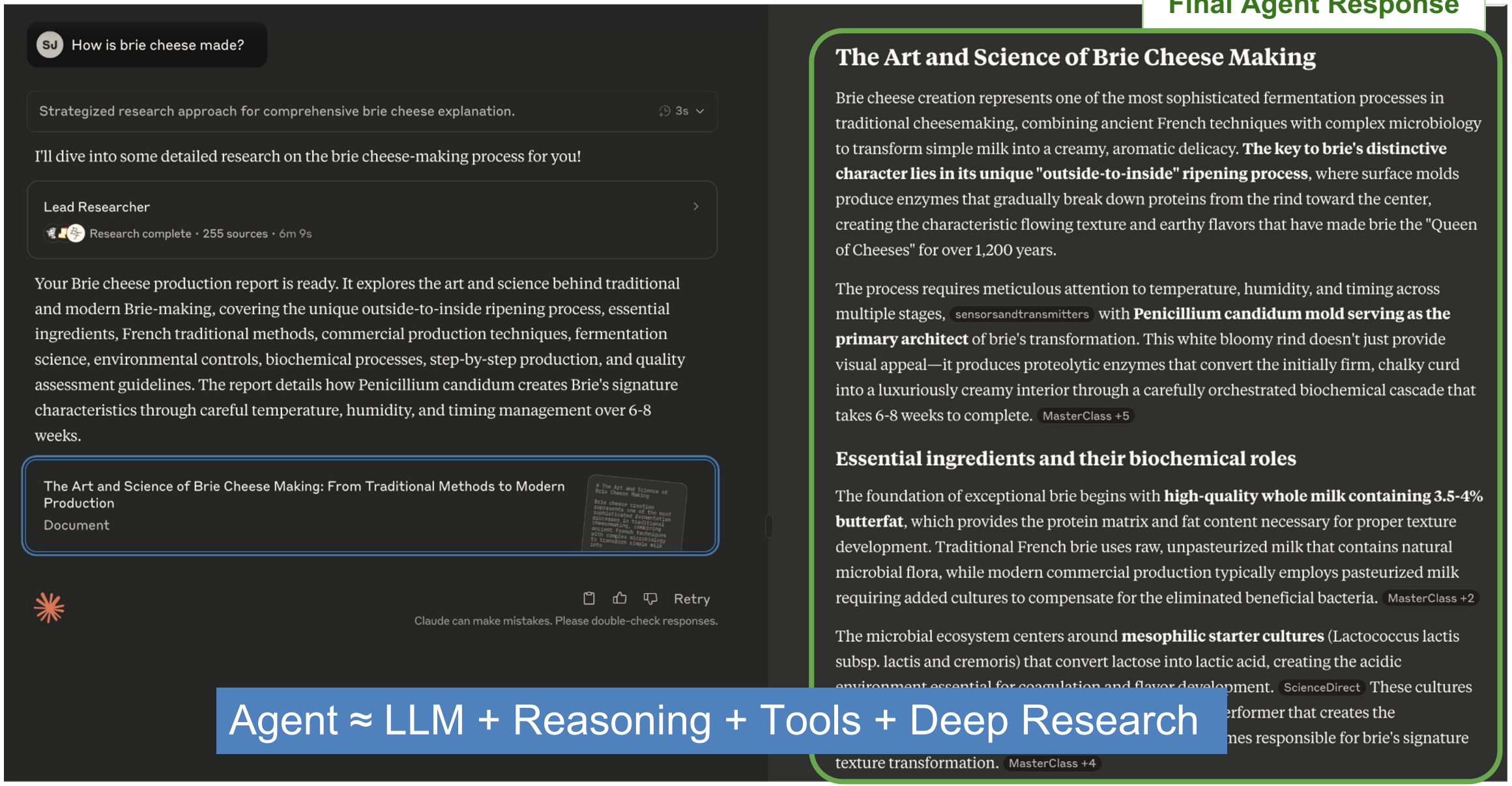
Web Search Agent



Agent Example: Claude



Agent Example: Claude



Final Agent Response

The Art and Science of Brie Cheese Making

Brie cheese creation represents one of the most sophisticated fermentation processes in traditional cheesemaking, combining ancient French techniques with complex microbiology to transform simple milk into a creamy, aromatic delicacy. **The key to brie's distinctive character lies in its unique "outside-to-inside" ripening process**, where surface molds produce enzymes that gradually break down proteins from the rind toward the center, creating the characteristic flowing texture and earthy flavors that have made brie the "Queen of Cheeses" for over 1,200 years.

The process requires meticulous attention to temperature, humidity, and timing across multiple stages, sensorsandtransmitters with **Penicillium candidum mold serving as the primary architect** of brie's transformation. This white bloomy rind doesn't just provide visual appeal—it produces proteolytic enzymes that convert the initially firm, chalky curd into a luxuriously creamy interior through a carefully orchestrated biochemical cascade that takes 6-8 weeks to complete. MasterClass +5

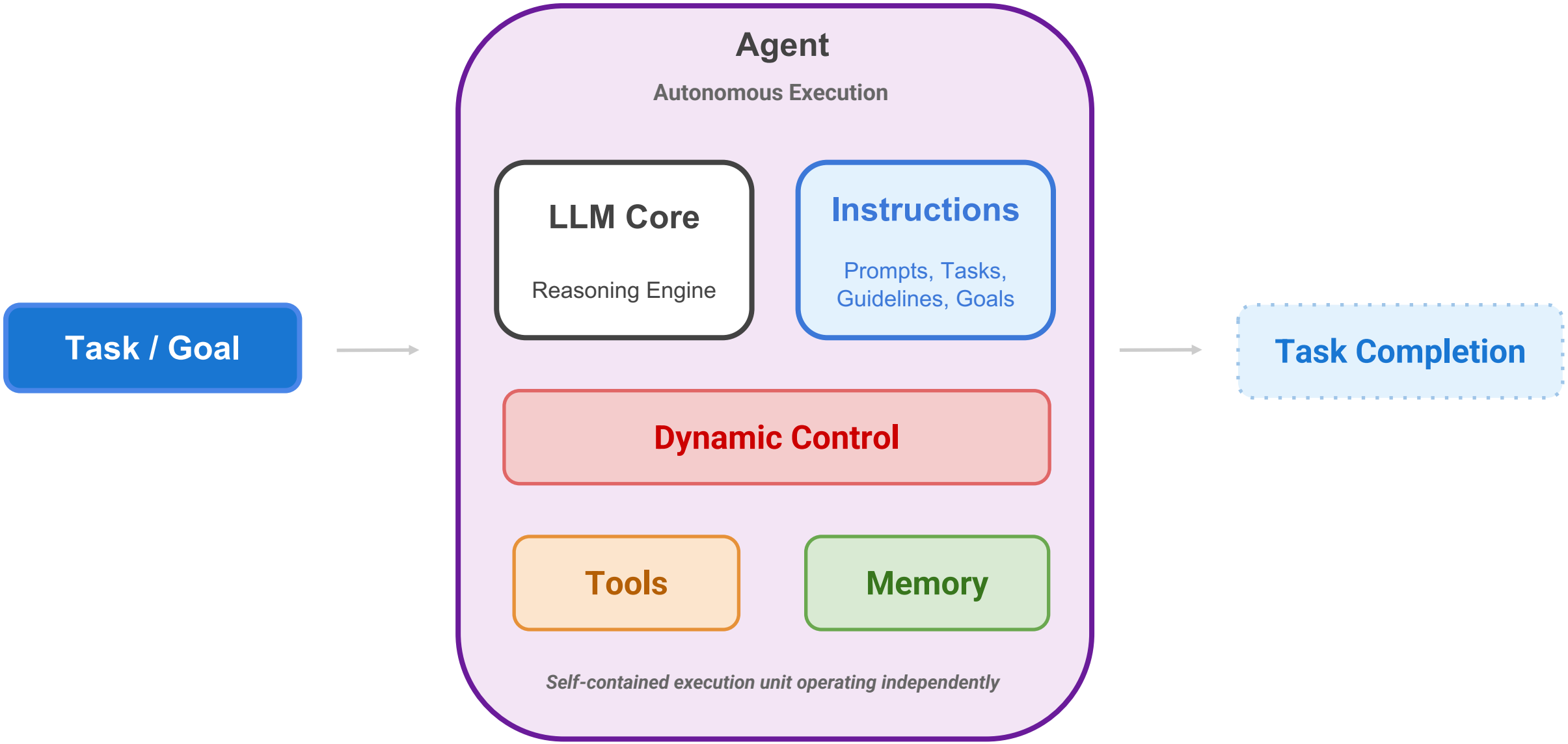
Essential ingredients and their biochemical roles

The foundation of exceptional brie begins with **high-quality whole milk containing 3.5-4% butterfat**, which provides the protein matrix and fat content necessary for proper texture development. Traditional French brie uses raw, unpasteurized milk that contains natural microbial flora, while modern commercial production typically employs pasteurized milk requiring added cultures to compensate for the eliminated beneficial bacteria. MasterClass +2

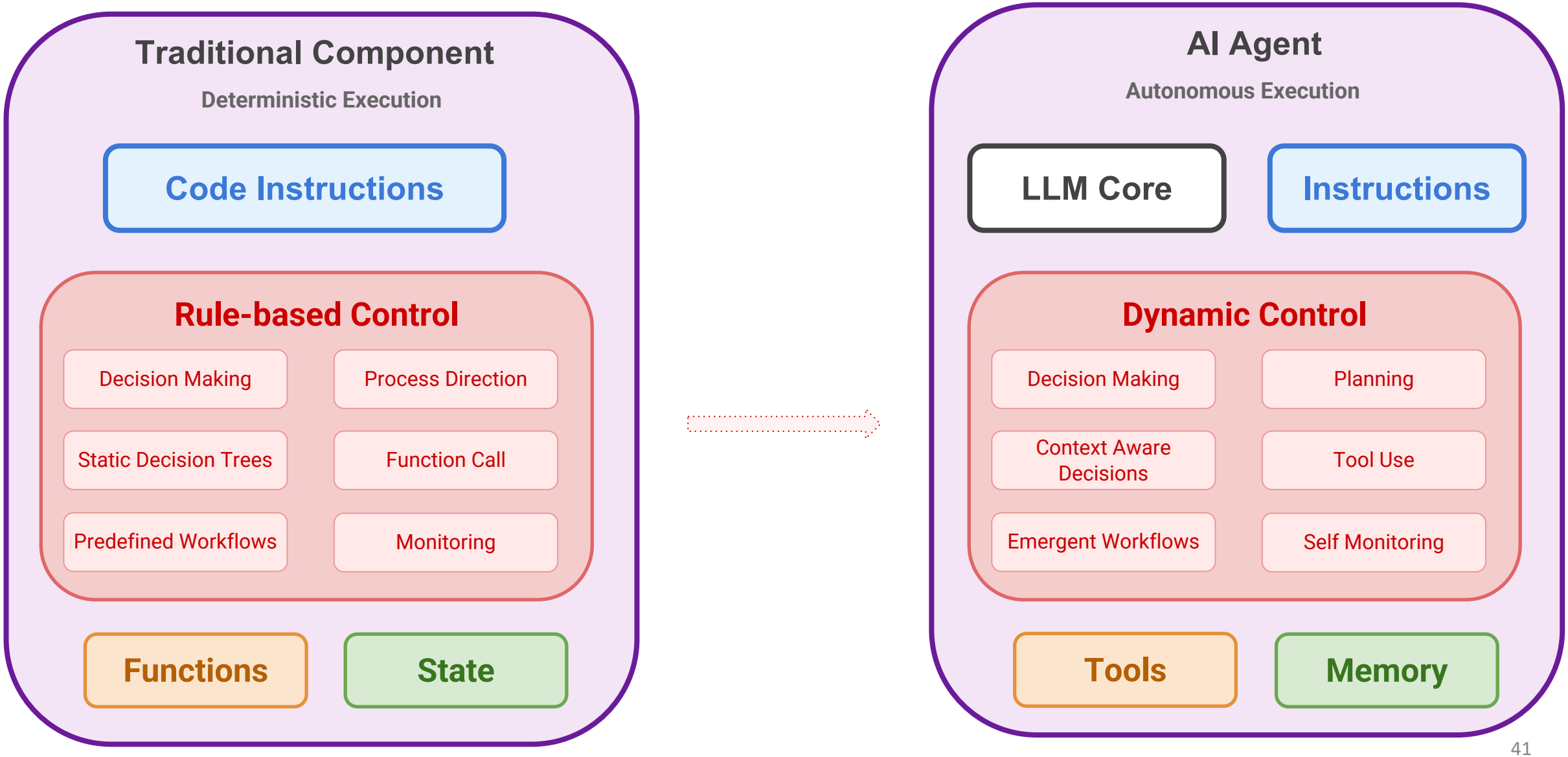
The microbial ecosystem centers around **mesophilic starter cultures** (Lactococcus lactis subsp. lactis and cremoris) that convert lactose into lactic acid, creating the acidic environment essential for coagulation and flavor development. ScienceDirect These cultures erformer that creates the mes responsible for brie's signature texture transformation. MasterClass +4

Agent ≈ LLM + Reasoning + Tools + Deep Research

Review: Agent



From Traditional to AI Agent Systems



The Autonomy Spectrum

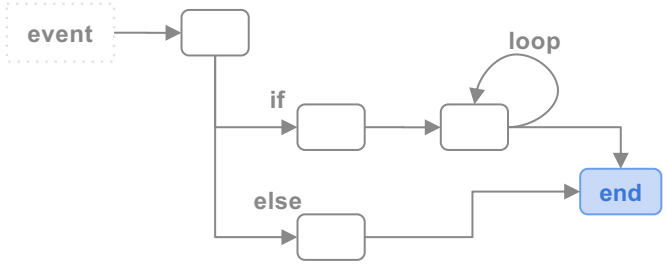
Traditional Component

Deterministic Execution

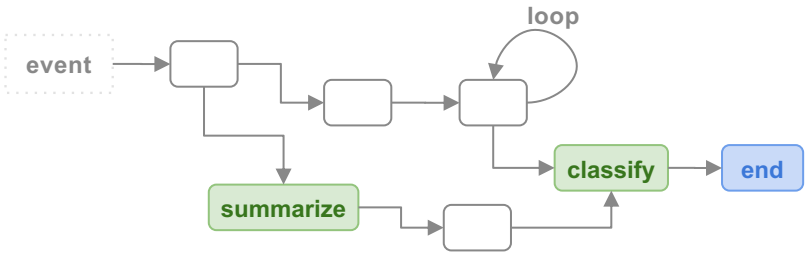
AI Agent

Autonomous Execution

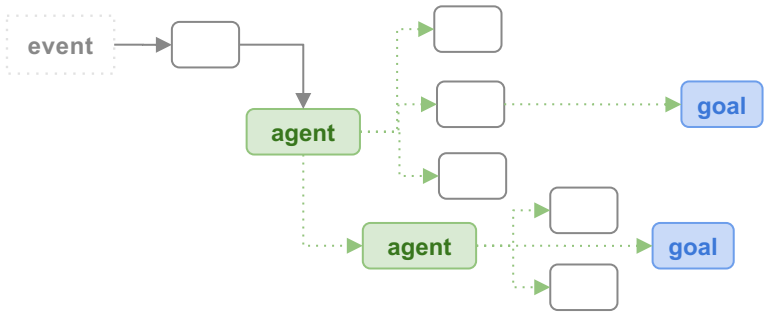
Directed Graphs



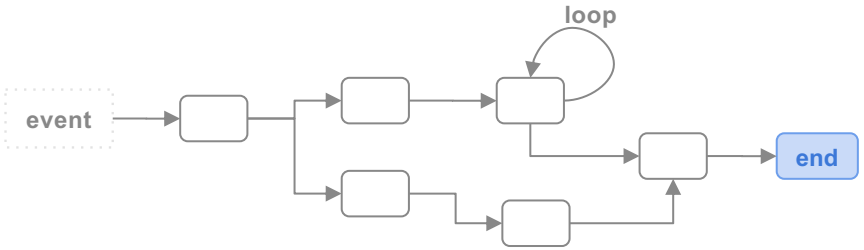
Workflow Orchestrators (with ML)



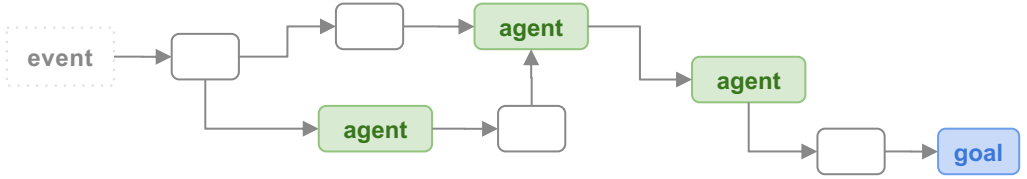
Autonomous Orchestrators



Workflow Orchestrators



Hybrid Orchestrators



Rule-based Systems

Predictable

Data Driven Workflows

ML-Enhanced Tasks

LLM-Enhanced Tasks

Structured Output

Guided / Bounded Tool Selection

Full Agent Autonomy

Agent Layers

Layer	Controls	Primary owner
LLM	What gets generated from the context it's given: the reasoning, the plan, the answer	LLM provider
Tool	What the agent can actually do in its environment: each capability's contract, inputs, outputs, side effects	Software Engineer
Context	What the model receives next: instructions, evidence, memory, tool results	AI Engineer
Agent Harness	How execution proceeds and what is permitted: the loop, tool dispatch, limits, checks	Platform / AI Engineer
Skills	Which reusable procedure the agent loads on demand	Domain expert

 For this class and small teams you will wear all hats

When multi-agent is overkill

Stay single-agent when

- One goal, one context — it fits in a single window
- The “specialists” would share the same tools and instructions
- Latency and cost matter: every extra agent is another model call
- You cannot yet evaluate the single agent — fix that first

Go multi-agent when

- Distinct skills that need different instructions, tools or models
- Independent sub-tasks that can run in parallel
- Context would overflow one window (research, long documents)
- Isolation: different permissions or trust boundaries per agent

Rule of thumb: **add an agent only when you can name what it decides that the orchestrator should not.**

More agents = more places for the loop to go wrong; each one needs its own evaluation.

THANK YOU